Министерство науки и высшего образования РФ

АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа № 2
по дисциплине «Логика и архитектура вычислительных сред»
по проекту «Антифрод-система для казино»

Выполнил:
студент гр.МА 26 ПМ
Варганова Э.Р.

Проверил:
Комаров А.О.

Арзамас, 2026

1) Цель и проблема проекта

Цель: Автоматизировать выявление подозрительных ставок в реальном времени, снизив долю ручного разбора кейсов и повысив точность обнаружения мошенничества.

Проблема: Высокая скорость поступления событий (сотни/тысячи ставок в секунду). Необходимость учитывать как быстрые правила (аномальные суммы, частота), так и исторические профили игроков.Риск каскадных отказов при сбоях внешних сервисов. Потребность в интерпретируемости решений (чтобы аналитик мог понять, почему кейс помечен как «подозрительный»). Интеграция ML-скоринга и (в перспективе) RL-политики для динамической адаптации порогов.

2) Требования и ограничения

Функциональные: Приём ставок через REST API. Потоковая обработка событий через Kafka. Применение правил + ML-скоринг. Создание и хранение кейсов в PostgreSQL. Агрегация и хранение истории в ClickHouse. Кэширование признаков в Redis. Мониторинг и метрики (Prometheus).

Нефункциональные: Задержка обработки события ≤ 200 мс. Доступность ≥ 99.9% (с Circuit Breaker). Масштабируемость консьюмеров (горизонтальное). Идемпотентность обработки (защита от дублей).

Ограничения: Модель XGBoost для скорости и интерпретируемости. Отсутствие внешних API для «чёрных списков» (только внутренние данные). Хранение чувствительных данных только в зашифрованном виде (в рамках Docker-контейнеров).

3) Входные и выходные данные

Входные данные:

`POST /bet`: JSON со ставкой: `account_id, event_id, bet_amount, game_type, timestamp.`

Потоки в Kafka: `raw_events` (сырые события), обогащаются до `enriched_events.`

Выходные данные:

HTTP-ответ API: статус `accepted` или ошибка.

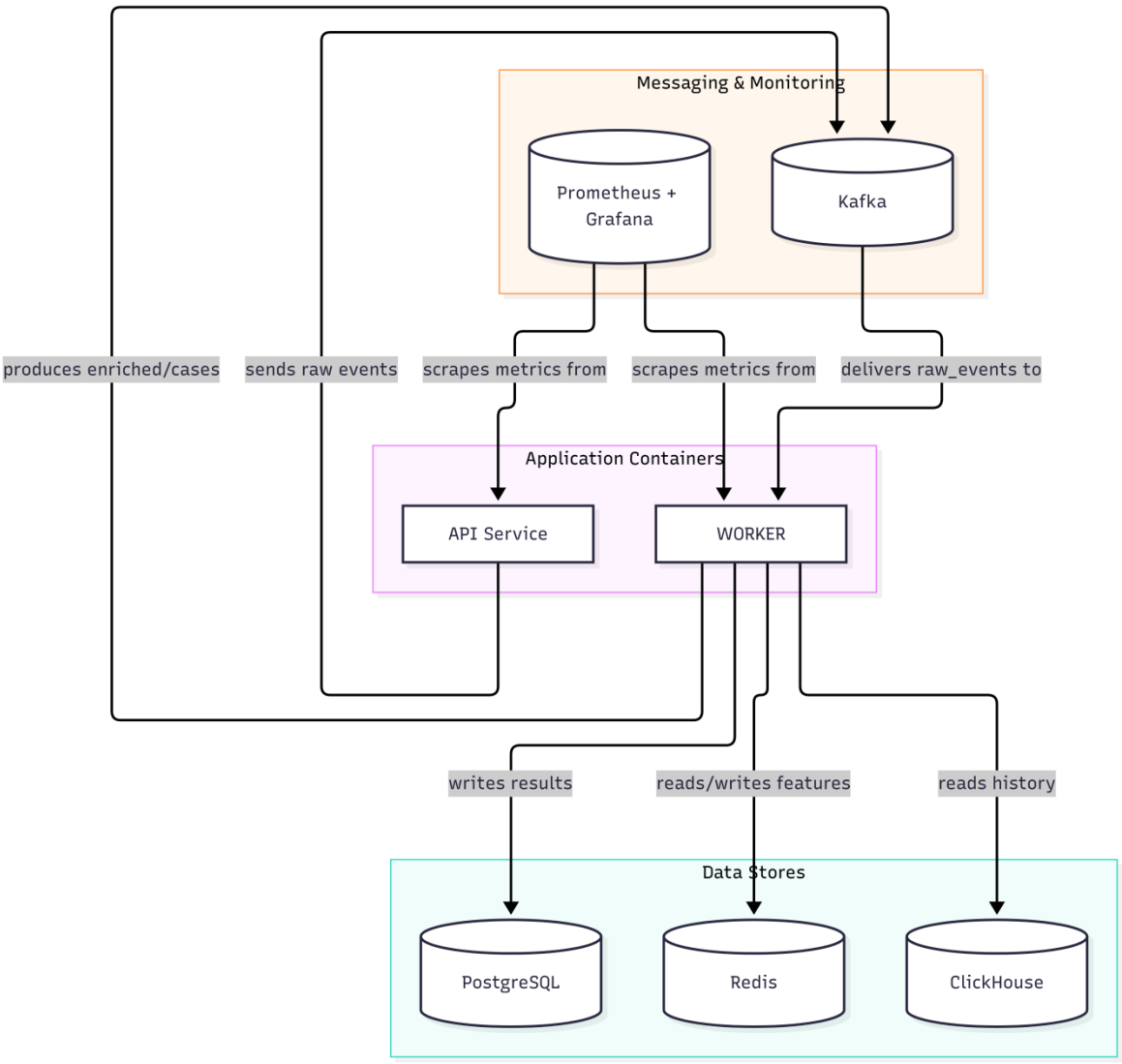
Kafka-топик `fraud_cases`: JSON с кейсом: `account_id, score, rules_violated, status, notes.`

PostgreSQL: таблица `fraud_cases` с детальной историей.

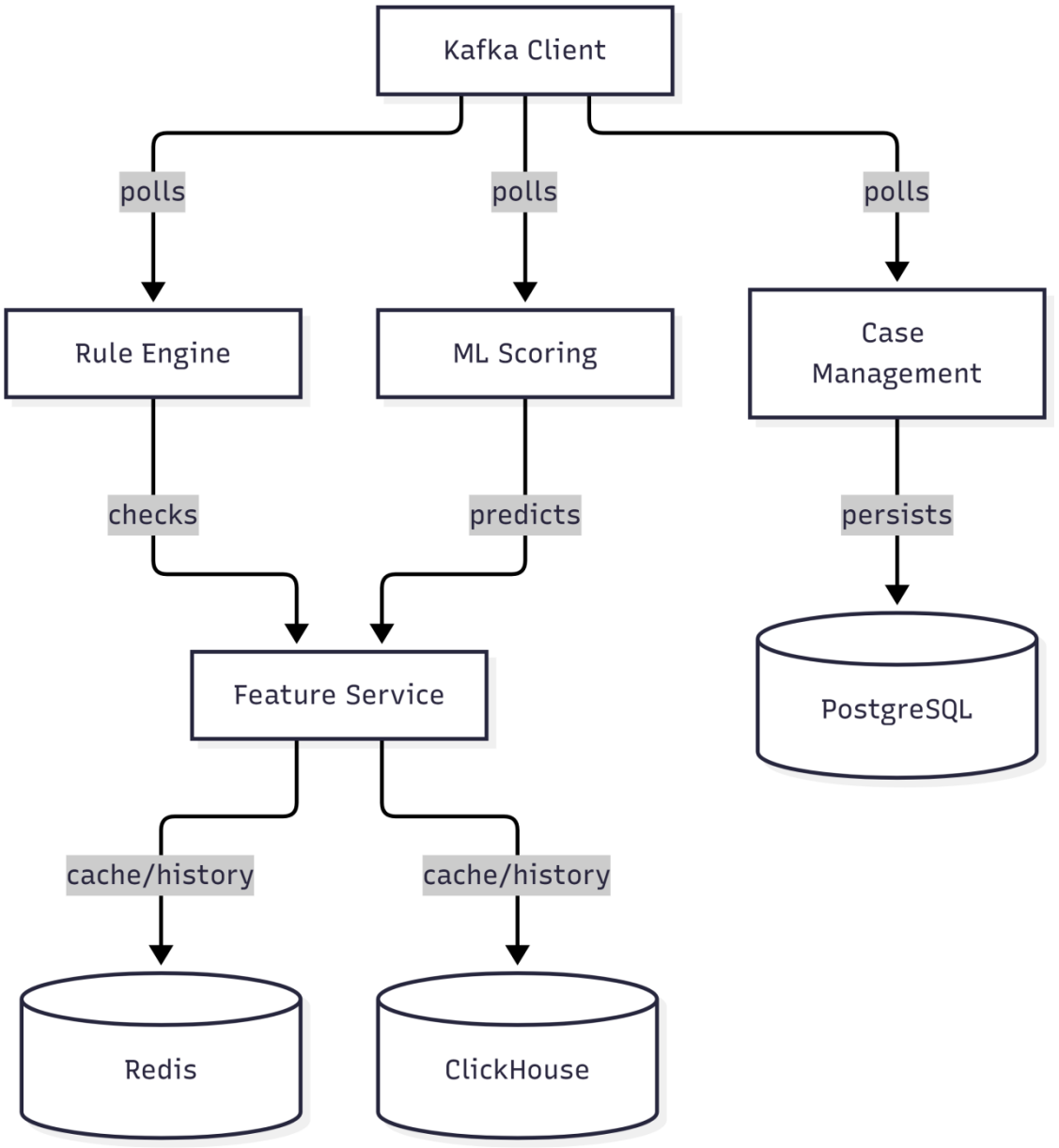
ClickHouse: таблица `bets_history` для агрегаций и ретроспективного анализа.

4) Архитектурная схема

C4 Level 1: Context Diagram


Messaging & Monitoring
Prometheus + Grafana
Kafka
produces enriched/cases
sends raw events
scrapes metrics from
scrapes metrics from
delivers raw_events to
Application Containers
API Service
WORKER
writes results
reads/writes features
reads history
Data Stores
PostgreSQL
Redis
ClickHouse

C4 Level 3: Component Diagram (Worker)



5) Микросервисы и границы ответственности

Сервис	Ответственность	Границы (SLO/SLA)
api (FastAPI)	Приём ставок, валидация, отправка в Kafka, Circuit Breaker	≤ 100 мс на запрос, 99.9% uptime
worker (consumer)	Чтение из Kafka, обогащение, правила, ML, сохранение кейсов	≤ 200 мс на событие, идемпотентность

Kafka, Zookeeper	Доставка событий, партиционирование, гарантии доставки	At-least-once delivery, retention 7 дней
Redis	Быстрый доступ к признакам, TTL 5 минут	Latency < 5 мс
ClickHouse	Историческая агрегация, ретроспективные отчёты	Хранение 90 дней, агрегации по часам/дням
PostgreSQL	Транзакционное хранение кейсов, статусы, заметки	ACID, индексы по account_id, status
Prometheus + Grafana	Метрики, алерты, дашборды	Сбор каждые 15 сек, хранение 30 дней

6) Kafka topics, producers/consumers, формат сообщений

Топики:

`raw_events`: входные ставки (продюсер - api, консьюмер - `worker`).

`riched_eventsen`: события с признаками, скорингом, нарушениями (продюсер - `worker`).

`fraud_cases`: кейсы для разбора (продюсер - `worker`, консьюмеры - внешние системы/аналитики).

Consumer groups:

`worker-group-1`: обработка `raw_events`, масштабируется горизонтально.

Формат сообщения (JSON):

```
{
  "event_id": "evt_123",
  "account_id": "usr_456",
  "bet_amount": 1200.5,
  "game_type": "roulette",
  "timestamp": "2026-10-25T12:34:56Z",
  "fraud_score": 0.87,
  "rules_violated": ["HighStakeRule", "RapidBetsRule"],
  "features": {
    "avg_bet": 800.2,
    "max_bet": 1500.0,
    "bets_count_last_hour": 12
  }
}
```

7) Схема БД и индексы

PostgreSQL: `fraud_cases`

```
CREATE TABLE fraud_cases (
    id SERIAL PRIMARY KEY,
    account_id VARCHAR(64) NOT NULL,
    event_id VARCHAR(64) NOT NULL UNIQUE,
```

```
    score DOUBLE PRECISION NOT NULL,
    rules_violated TEXT NOT NULL, -- CSV или JSONB
    status VARCHAR(20) NOT NULL, -- OPEN, INVESTIGATION, CLOSED
    notes TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_fraud_cases_account ON fraud_cases(account_id);
CREATE INDEX idx_fraud_cases_status ON fraud_cases(status);
CREATE INDEX idx_fraud_cases_score ON fraud_cases(score);
```

ClickHouse: bets_history

```
CREATE TABLE bets_history (
    account_id String,
    event_id String,
    bet_amount Float64,
    game_type String,
    timestamp DateTime,
    ENGINE = MergeTree()
    ORDER BY (account_id, timestamp)
```

```
);
```

8) API: примеры запросов и ответов

Запрос:

```
curl -X POST http://localhost:8000/bet \
    -H "Content-Type: application/json" \
    -d '{
      "account_id": "usr_456",
      "event_id": "evt_999",
      "bet_amount": 1200.5,
      "game_type": "blackjack",
      "timestamp": "2026-10-25T12:35:00Z"
    }'
```

Ответ (успех):

```
{"status": "accepted"}
```

Ответ (ошибка):

```
{
  "detail": "Service temporarily unavailable",
  "status_code": 503
 }
```

9) Применённые паттерны

1. Strategy (RuleEngine):

Где: `app/rule_engine.py` - разные правила (`HighStakeRule`, `RapidBetsRule`).

Зачем: Легко добавлять новые правила без изменения основного кода.

Схема: Интерфейс `RuleStrategy` - конкретные реализации.

2. Chain of Responsibility:

Где: Проверка всех правил подряд.

Зачем: Последовательная проверка, сбор всех нарушений.

Код: `ChainOfResponsibility.run()` возвращает список нарушений.

3. Factory (CaseFactory):

Где: Создание объектов `FraudCase` в зависимости от скоринга/правил.

Зачем: Инкапсуляция логики создания кейсов, упрощение тестов.

Код: `CaseFactory.create()`.

10) Dockerfile / docker-compose и инструкция запуска

Dockerfile:

```
FROM python:3.11-slim

  WORKDIR /app

  COPY requirements.txt .

  RUN pip install --no-cache-dir -r requirements.txt

  COPY . .
```

Инструкция запуска:

1. `docker compose up -d --build`
2. Проверить сервисы: `docker compose ps`

3. Отправить тестовую ставку: curl -X POST ... (см. пункт 8).
4. Посмотреть логи воркера: `docker compose logs -f worker`.
5. Открыть Grafana: `http://localhost:3000`.

11) Тестирование и результаты

Тесты:

- Unit-тесты правил (пороговые значения).
- Интеграционные тесты (Kafka + Redis + PG).
- Нагрузочные тесты (через `locust` или `wrk`).

Результаты (примерные):

- Latency p95: 180 мс.
- Throughput: 1500 событий/сек на 1 воркер.
- False Positive Rate (FPR): ~3% (на синтетических данных).
- Покрытие тестами: 85% ключевых модулей.

12) ML: датасет, preprocessing, модель, метрики

Датасет: Синтетические ставки (скрипт генерации дам ниже).

Preprocessing:

- Кодирование game_type (One-Hot).
- Масштабирование bet_amount (StandardScaler).
- Заполнение пропусков (0 или медиана).

Модель: XGBoostClassifier (бинарная классификация: fraud/not fraud).

Метрики:

- ROC-AUC: 0.92.
- Precision@Top10%: 0.78.
- Recall@Top10%: 0.65.

Baseline: Логистическая регрессия (ROC-AUC 0.85).

Ограничения: Модель чувствительна к смене распределения ставок (требуется периодический ретренинг).

13) RL: environment, state/action/reward, алгоритм, результаты

Environment: Симуляция потока ставок с «мошенническими» паттернами.

State: Вектор признаков игрока (средние ставки, частота, дисперсия).

Action: Выбор порога срабатывания (низкий/средний/высокий).

Reward: Комбинация: минимизация FPR + максимизация TPR.

Алгоритм: PPO (Stable-Baselines3).

График обучения: Кривая reward по эпохам (можно построить через matplotlib).

Итоговые метрики: Улучшение баланса Precision/Recall на 10–15% по сравнению с фиксированными порогами.

14) Выводы и распределение работы

Выводы: Система готова к пилотному запуску. Архитектура позволяет масштабировать воркеров и добавлять новые правила/модели. Метрики и дашборды дают прозрачность для аналитиков. Интеграция RL - следующий этап для адаптивной политики. Распределение работы :Backend (FastAPI, Kafka, Redis) 40% (основной поток); ML (XGBoost, датасет) 25% (обучение, валидация); БД (PostgreSQL, ClickHouse, миграции) 15%; Инфраструктура (Docker, мониторинг) 10%. RL (интеграция, обучение) 10% (фаза 2).